AeroCore: Flight Booking Driven by a Five-Level MLFQ Scheduler

Operating Systems project report · 27 October 2026

1. Introduction

AeroCore is a flight-booking program in C where an operating-system scheduler decides which booking request is served first, and a crash-safe journal stores the result.

CPU scheduling. The scheduler picks which ready process gets the CPU next. A multilevel feedback queue (MLFQ) keeps several queues of different priority and moves a process between them by its behaviour. New processes start in the top queue, processes that use a lot of CPU sink, and processes that wait too long are promoted (aging). Variants of MLFQ are the basis of the schedulers in general-purpose systems such as BSD, Solaris and Windows.

Journaling. A journal (write-ahead log) appends each change as a record that only counts once it is completely on disk. A crash can then never leave a half-applied change. File systems such as ext4 and NTFS and most databases rely on this idea.

This project joins the two. Every booking, cancellation, check-in and refund is treated as a process. Requests are queued, run through a five-queue MLFQ on a simulated CPU, and committed to the journal only when the scheduler finishes them. Scheduling order therefore decides who wins a contested seat.

Topics covered:

- Five queues, each with its own algorithm: Round Robin, SJF, Priority, SRTF and FCFS
- Preemption, demotion and aging, plus FCFS, SJF and RR baselines for comparison
- Booking rules: seat map, duplicate prevention, PNR life cycle, 80% refund
- Append-only journal with checksums and recovery from a torn write

2. Work

AeroCore turns a booking system into a scheduling experiment: the order in which the scheduler finishes requests decides which one succeeds.

Idea

1. A command (book, cancel, checkin, refund) becomes a request with an arrival tick, a burst (CPU ticks needed) and a priority.
2. `run` plays all pending requests through the five-queue MLFQ, one tick at a time.
3. Requests are applied to the database in the order they finish, not the order they arrived.
4. Each change is one checksummed journal line, so a crash loses at most the write in progress.

Scheduler design

Queue	Algorithm	Picks next by	CPU allotment before demotion
Q1	Round Robin, quantum 2	urgent first, then entry order	4 ticks
Q2	SJF	smallest total burst	4 ticks
Q3	Priority	smallest priority number	6 ticks
Q4	SRTF	smallest remaining time	8 ticks
Q5	FCFS	earliest entry	none

- **Enter and demote:** every request starts in Q1; using up a queue's allotment moves it down one level.
- **Preempt:** a ready process in a higher queue always takes the CPU from a lower one.
- **Age:** waiting 25 ticks (default) without the CPU moves a request up one queue and lowers its priority number by 1.

Request defaults

Request	Burst (ticks)	Priority (1 = most important)
BOOK	6	3
CANCEL	3	4
CHECKIN	2	2
REFUND	8	5

The `urgent` option forces priority 1. Burst and arrival can be set with `b=<n>` and `@<tick>`.

Booking rules

- Each flight has a 10 × 4 seat map (rows 1-10, columns A-D). A seat is taken while its booking is BOOKED or CHECKED_IN.
- A seat cannot be double-booked, and a passenger cannot hold two live bookings on one flight. This is checked when the command is typed and again when the scheduler commits it.
- Status flow: BOOKED → CHECKED_IN, and BOOKED → CANCELLED → REFUNDED. A refund returns 80% of the fare. A checked-in booking cannot be cancelled.

Constraints

- Time is simulated in ticks, not measured CPU time. Context switches cost zero ticks.
- Fixed capacities: 64 requests per batch, 256 passengers, 1024 bookings.
- The journal is never compacted, and there are no concurrent writers.
- Each `run` is a separate epoch that starts at tick 0.

3. Implementation

The program is about 900 lines of C in four files: `aerocore.h` (shared types), `main.c` (command line), `sched.c` (scheduler) and `airline.c` (bookings and journal). It builds with `make` and uses only the standard library. The excerpts are real project code, shortened where marked `...`; the comments explain each step.

3.1 aerocore.h: one request is one process

```
enum { RR, SJF, PRIO, SRTF, FCFS };               /* algorithm used by each queue */
enum { BOOK, CANCEL, CHECKIN, REFUND };           /* request types */
enum { BOOKED, CHECKED_IN, CANCELLED, REFUNDED }; /* booking status */

/* One request == one scheduler process */
typedef struct {
    int id, type, urgent, arrival, burst, prio;   /* input: what the user asked for */
    int pax; char flight[8], seat[4], pnr[8];     /* BOOK uses pax/flight/seat, the rest use pnr */
    int rem, q, seq, q_used, slice, wait_q,       /* scheduler state: ticks left, current queue,  */
        first, finish;                            /* entry order, CPU used here, RR slice, wait;  */
                                                  /* first/finish feed the metrics                */
    char msg[96];                                 /* outcome text, filled in by apply_req() */
} Req;

/* nq queues; queue i runs alg[i]; a job leaves queue i after allot[i] CPU ticks */
typedef struct { int nq, alg[5], allot[5], quantum, age, trace, urgent_first; char name[16]; } Cfg;
```

3.2 main.c: from command to request, and commit order

```
static const int BURST[] = {6, 3, 2, 8};   /* CPU ticks per type: BOOK, CANCEL, CHECKIN, REFUND */
static const int PRIOR[] = {3, 4, 2, 5};   /* 1 = most important; urgent forces 1 */

static void submit(Req r) {                /* turn a typed command into a scheduler process */
    r.id = np + 1;
    if (r.arrival < 0) r.arrival = np;     /* default: one new request per tick */
    if (r.burst <= 0) r.burst = BURST[r.type];
    r.prio = r.urgent ? 1 : PRIOR[r.type];
    pend[np++] = r;                        /* waits in the pending list until 'run' */
}

static void run(void) {
    Cfg c = cfg_mlfq(quantum, age);
    sched_run(pend, np, &c, &s);           /* 1. simulate: fills in every request's finish tick */

    int o[MAXREQ];                         /* 2. sort request indexes by finish tick */
    for (int i = 0; i < s.n; i++) {
        int j = i;
        while (j && s.p[o[j - 1]].finish > s.p[i].finish) { o[j] = o[j - 1]; j--; }
        o[j] = i;
    }
    for (int k = 0; k < s.n; k++) {        /* 3. commit in completion order: the scheduler */
        Req *r = &s.p[o[k]];               /*    decides who wins a contested seat */
        apply_req(r);                      /* writes the journal, sets r->msg */
        /* ... print one result line ... */
    }
}
```

3.3 sched.c: the MLFQ engine

```
Cfg cfg_mlfq(int quantum, int age) {       /* the five queues and their CPU allotments */
    return (Cfg){5, {RR, SJF, PRIO, SRTF, FCFS}, {4, 4, 6, 8, INF}, quantum, age, 0, 1, "MLFQ-5Q"};
}

/* Smaller key runs first; ties go to the earliest queue entry (seq). FCFS uses seq only. */
static int key(const Req *r, int alg, const Cfg *c) {
    return alg == SJF ? r->burst : alg == PRIO ? r->prio : alg == SRTF ? r->rem
         : alg == RR && c->urgent_first ? !r->urgent : 0;   /* RR queue: urgent first */
}
```

The main loop runs once per tick. Queues are not separate data structures: each request carries its queue number `q` and an entry counter `seq`, and choosing a job is a scan.

```
for (int t = 0; done < n && t < 100000; t++) {              /* one iteration = one tick */
    /* 1. arrivals join the tail of Q1 */
    for (int i = 0; i < n; i++)
        if (p[i].arrival == t) p[i].seq = seqc++;
    if (rq >= 0) { p[rq].seq = seqc++; rq = -1; }           /* last tick's slice-end/demoted job re-enters */

    /* 2. aging: a waiting job that reaches the limit moves up one queue */
    if (c->age)
        for (int i = 0; i < n; i++) {
            Req *r = &p[i];
            if (r->seq < 0 || !r->rem || i == cur || r->q == 0) continue;   /* skip: not arrived, done, running, in Q1
*/
            if (++r->wait_q >= c->age) {
                r->q--; r->q_used = r->wait_q = 0; r->seq = seqc++;       /* promote, join the tail */
                if (r->prio > 1) r->prio--;                              /* priority number drops by 1 */
                s->npromo++;
            }
        }

    /* 3. highest non-empty queue wins */
    int top = c->nq;
    for (int i = 0; i < n; i++)
        if (p[i].seq >= 0 && p[i].rem && p[i].q < top) top = p[i].q;
    if (top == c->nq) { /* nothing ready: idle tick */ continue; }

    /* 4. pick the process inside that queue */
    int alg = c->alg[top], nxt = -1;
    if (cur >= 0 && p[cur].q == top && alg != SRTF) nxt = cur;   /* non-preemptive / RR slice: keep the CPU */
    else {
        for (int i = 0; i < n; i++) {                           /* scan: smallest key(), then smaller seq */
            Req *r = &p[i];
            if (r->seq < 0 || !r->rem || r->q != top) continue;
            if (nxt < 0 || key(r, alg, c) < key(&p[nxt], alg, c) ||
                (key(r, alg, c) == key(&p[nxt], alg, c) && r->seq < p[nxt].seq)) nxt = i;
        }
        if (alg == SRTF && cur >= 0 && p[cur].q == top &&         /* SRTF: switch only if strictly shorter */
            key(&p[nxt], alg, c) >= key(&p[cur], alg, c)) nxt = cur;
    }
    if (nxt != cur) {
        if (cur >= 0) s->npreempt++;                            /* old job still had work: it was preempted */
        p[nxt].slice = 0;
    }
    cur = nxt;

    /* 5. run the chosen job for one tick */
    Req *r = &p[cur]; int ci = cur;
    r->rem--; r->q_used++; r->slice++; r->wait_q = 0; s->busy++;
    if (!r->rem) { r->finish = t + 1; done++; cur = -1; }       /* finished */
    else {
        if (alg == RR && r->slice >= c->quantum)                /* RR slice used up: go to the tail */
            { r->slice = 0; cur = -1; rq = ci; }
        if (r->q < c->nq - 1 && r->q_used >= c->allot[r->q]) {   /* allotment used up: demote */
            r->q++; r->q_used = r->wait_q = 0;
            cur = -1; rq = ci; s->ndemote++;
        }
    }
}
```

Metrics are computed afterwards from each request's arrival, first-run and finish ticks:

```
m.wait += r->finish - r->arrival - r->burst;   /* time ready but not running */
m.tat  += r->finish - r->arrival;              /* turnaround */
m.resp += r->first - r->arrival;               /* time until first CPU tick */
/* after the loop: divide by n; throughput = n / makespan; CPU% = 100 * busy / makespan */
```

3.4 airline.c: booking rules and the commit path

```
const char *book_check(int id, const char *f, const char *seat) {
    if (!pax(id)) return "unknown passenger";
    if (!flight(f)) return "unknown flight";
    if (holder(f, seat)) return "seat already taken";           /* an active booking holds the seat */
    if (pax_active_pnr(id, f)) return "passenger already holds a booking on this flight";
```

```
    return 0;                                                  /* NULL = ok */
}

int apply_req(Req *r) {            /* the only function that changes a booking */
    if (r->type == BOOK) {
        const char *err = book_check(r->pax, r->flight, r->seat);   /* checked again at commit time */
        if (err) return fail(r, err);
        /* ... new PNR, status BOOKED, fare from the flight ... */
    } else {
        Booking *e = booking(r->pnr);
        if (!e) return fail(r, "unknown PNR");
        b = *e;
        int need = r->type == REFUND ? CANCELLED : BOOKED;          /* status the booking must be in */
        if (b.status != need) return fail(r, why);
        if (r->type == CANCEL)       b.status = CANCELLED;          /* seat is released automatically */
        else if (r->type == CHECKIN) b.status = CHECKED_IN;
        else { b.status = REFUNDED; amt = -(b.fare * 80 / 100); }   /* refund 80% of the fare */
    }
    if (!commit(&b, r->type, amt)) return fail(r, "journal write failed, nothing changed");
    return 1;
}
```

3.5 airline.c: the journal and crash recovery

Each record is one line, `record*hash`. The hash is a 32-bit FNV-1a checksum of the text before the star.

```
static unsigned fnv(const char *s) {            /* FNV-1a checksum */
    unsigned h = 2166136261u;
    for (; *s; s++) h = (h ^ (unsigned char)*s) * 16777619u;
    return h;
}

static int jwrite(const char *body) {
    char line[440];
    int n = snprintf(line, sizeof line, "%s*%08x\n", body, fnv(body));   /* record*hash + newline */
    int torn = crash_in == 0;                   /* crash demo: this write will be torn */
    if (crash_in > 0) crash_in--;
    size_t len = torn ? n / 2 : n;              /* torn = half a line, no newline */
    if (fwrite(line, 1, len, jf) != len || fflush(jf) || fsync(fileno(jf))) return 0;   /* force to disk */
    if (torn) { puts("*** simulated power loss mid-write ***"); fflush(stdout); _exit(137); }
    return 1;
}

static int put(const char *fmt, ...) {         /* journal first, then update memory */
    /* ... format the record into body ... */
    return jwrite(body) && replay(body);        /* replay() is the same code used at startup */
}
```

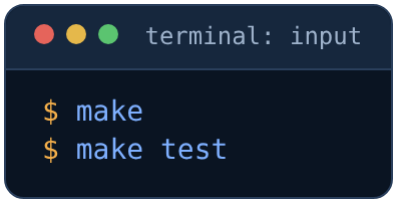
At startup the journal is replayed. A line counts only if it ends in a newline and its checksum matches; everything else is discarded and the file is cut back to the last good record.

```
while (fgets(line, sizeof line, f)) {
    size_t n = strlen(line);
    char *star = strrchr(line, '*');
    unsigned ck;
    if (line[n - 1] != '\n' || !star || sscanf(star + 1, "%8x", &ck) != 1)
        { (*discarded)++; continue; }           /* torn line: no newline or no checksum */
    *star = 0;
    if (fnv(line) != ck || !replay(line))
        { (*discarded)++; continue; }           /* damaged line: checksum mismatch */
    (*replayed)++; good = ftell(f);             /* offset just after the last good line */
}
fclose(f);
if (truncate(path, good)) return 0;             /* cut the torn tail so new appends start clean */
```

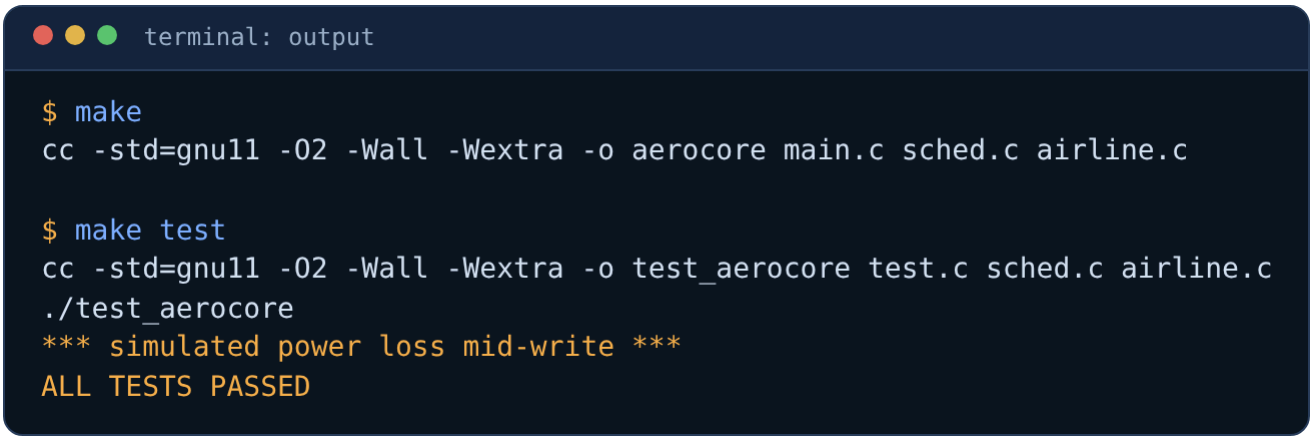
4. Outputs

Each experiment shows its **input** (the commands given to the program) and its **output** (the program's reply), as terminal screenshots rendered from real runs of the compiled program. Typed commands follow the `aerocore>` prompt. PNR codes and timestamps are generated per run, so they differ on every execution; the life-cycle experiment reuses the PNRs printed in experiment 2.

Experiment 1: build and tests (Makefile, test.c)



Input 1: the two commands



Output 1: both programs compile without warnings; the power-loss line is printed on purpose by the crash test, and the last line is the result

Experiment 2: queuing and committing bookings (main.c, sched.c, airline.c)

input_book.txt (passed to ./aerocore)

```
$ cat input_book.txt
flights
pax add 0300-1234567 Ayesha Khan
pax add 0300-7654321 Bilal Ahmed
book 1 AC101
3B
book 2 AC101 3B
book 2 AC202 1A urgent
pending
run
flights
seats AC101
pax
history 2
txns
quit
```

Input 2: commands passed to the program

```
terminal: output (queuing requests)

AeroCore | aerocore.journal: 0 records recovered, 0 torn/damaged discarded | 'help' for commands
aerocore> flights
Flight  Route     Departure          Fare   Taken
AC101   KHI->DXB  2026-11-01 08:30  $250   0/40
AC202   LHE->IST  2026-11-01 14:10  $420   0/40
AC303   ISB->LHR  2026-11-02 02:45  $610   0/40
aerocore> pax add 0300-1234567 Ayesha Khan
passenger #1 created
aerocore> pax add 0300-7654321 Bilal Ahmed
passenger #2 created
aerocore> book 1 AC101
AC101 KHI->DXB  2026-11-01 08:30  fare $250  (0/40 taken)
      A  B  C  D
  1  [.][.][.][.]
  2  [.][.][.][.]
  3  [.][.][.][.]
  4  [.][.][.][.]
  5  [.][.][.][.]
  6  [.][.][.][.]
  7  [.][.][.][.]
  8  [.][.][.][.]
  9  [.][.][.][.]
 10  [.][.][.][.]
.=free  X=booked  #=checked in
Pick a seat (e.g. 3B): 3B
queued REQ 1: BOOK     pax 1 AC101 3B         arrival t=0   burst 6  prio 3
aerocore> book 2 AC101 3B
queued REQ 2: BOOK     pax 2 AC101 3B         arrival t=1   burst 6  prio 3
aerocore> book 2 AC202 1A urgent
queued REQ 3: BOOK     pax 2 AC202 1A         arrival t=2   burst 6  prio 1 URGENT
aerocore> pending
  REQ 1  BOOK     pax 1 AC101 3B         arrival 0   burst 6  prio 3
  REQ 2  BOOK     pax 2 AC101 3B         arrival 1   burst 6  prio 3
  REQ 3  BOOK     pax 2 AC202 1A         arrival 2   burst 6  prio 1 URGENT
```

Output 2a: requests are queued with defaults filled in. REQ 1 and REQ 2 want the same seat (3B on AC101). The seat map is the interactive picker.

```
terminal: output (run)

aerocore> run

Committing in scheduler completion order:
  [t=14 ] REQ 3  BOOK     pax 2 AC202 1A         PNR XG85ZK seat 1A confirmed, charged $420
  [t=16 ] REQ 1  BOOK     pax 1 AC101 3B         PNR BGLH29 seat 3B confirmed, charged $250
  [t=18 ] REQ 2  BOOK     pax 2 AC101 3B         FAILED: seat already taken

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  BOOK     -     0     6         3     0     16   10   16    0
  2  BOOK     -     1     6         3     6     18   11   17    5
  3  BOOK     yes   2     6         1     2     14    6   12    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1  |P3      |P2  |P1  |P2  |P3  |P1  |P2  |
|Q1  |Q1      |Q1  |Q1  |Q1  |Q2  |Q2  |Q2  |
0    2        6    8    10   12   14   16   18

Queue transitions / preemptions:
  t=6   P3 Q1->Q2 demoted (used 4 ticks)
  t=10  P1 Q1->Q2 demoted (used 4 ticks)
  t=12  P2 Q1->Q2 demoted (used 4 ticks)

MLFQ-5Q: avg wait 9.00 | avg turnaround 15.00 | avg response 1.67 | throughput 0.167 req/tick | CPU 100.0%
promotions 0, demotions 3, preemptions 0
```

Output 2b: run. The urgent REQ 3 commits first. REQ 1 finishes at t=16 and takes seat 3B; REQ 2 finishes at t=18 and fails with "seat already taken". The Gantt chart, queue changes and metrics follow the commit order.

terminal: output (flights, seats, history, txns)

```
aerocore> flights
Flight  Route     Departure          Fare   Taken
AC101   KHI->DXB  2026-11-01 08:30  $250   1/40
AC202   LHE->IST  2026-11-01 14:10  $420   1/40
AC303   ISB->LHR  2026-11-02 02:45  $610   0/40
aerocore> seats AC101
AC101 KHI->DXB  2026-11-01 08:30  fare $250  (1/40 taken)
      A  B  C  D
  1  [.][.][.][.]
  2  [.][.][.][.]
  3  [.][X][.][.]
  4  [.][.][.][.]
  5  [.][.][.][.]
  6  [.][.][.][.]
  7  [.][.][.][.]
  8  [.][.][.][.]
  9  [.][.][.][.]
 10  [.][.][.][.]
.=free  X=booked  #=checked in
aerocore> pax
#1   Ayesha Khan              0300-1234567
#2   Bilal Ahmed              0300-7654321
aerocore> history 2
Passenger #2 Bilal Ahmed (0300-7654321)
  PNR XG85ZK  AC202 seat 1A  BOOKED     fare $420
      tx#1    10-27 05:23:10  BOOK     +420
aerocore> txns
  XG85ZK      tx#1    10-27 05:23:10  BOOK     +420
  BGLH29      tx#2    10-27 05:23:10  BOOK     +250
```

Output 2c: state after the run. AC101 and AC202 show 1/40 taken, seat 3B is marked X, and the passenger history and transaction log show the committed bookings.

Experiment 3: scheduler on the 13-request demo and comparison (sched.c)

input_demo.txt (passed to ./aerocore)

```
$ cat input_demo.txt
demo
run
compare
quit
```

Input 3: commands passed to the program

```
terminal: output (demo)

AeroCore | aerocore.journal: 0 records recovered, 0 torn/damaged discarded | 'help' for commands
aerocore> demo
queued REQ 1: BOOK     pax 6 AC101 3A         arrival t=0   burst 24 prio 3
queued REQ 2: REFUND   PNR JCTGWM             arrival t=1   burst 28 prio 5
queued REQ 3: CHECKIN  PNR XU4G3L             arrival t=2   burst 2  prio 1 URGENT
queued REQ 4: CANCEL   PNR L7D28G             arrival t=3   burst 3  prio 4
queued REQ 5: BOOK     pax 7 AC101 3A         arrival t=4   burst 6  prio 3
queued REQ 6: BOOK     pax 8 AC202 5C         arrival t=5   burst 5  prio 3
queued REQ 7: CHECKIN  PNR R2VEEP             arrival t=6   burst 2  prio 2
queued REQ 8: CANCEL   PNR 89CA7R             arrival t=8   burst 3  prio 4
queued REQ 9: CHECKIN  PNR 89CA7R             arrival t=9   burst 2  prio 2
queued REQ 10: BOOK     pax 1 AC202 1A         arrival t=10  burst 7  prio 3
queued REQ 11: BOOK     pax 2 AC303 2D         arrival t=12  burst 14 prio 3
queued REQ 12: BOOK     pax 3 AC303 2D         arrival t=14  burst 4  prio 3
queued REQ 13: BOOK     pax 4 AC202 2B         arrival t=50  burst 3  prio 1 URGENT
demo batch queued: try 'run', then 'compare'
```

Output 3a: demo loads 8 passengers, 5 bookings and queues 13 requests, including two long jobs, two urgent requests, two seat races and a cancel against a check-in

```
terminal: output (run: commit order and request table)

aerocore> run

Committing in scheduler completion order:
  [t=4  ] REQ 3  CHECKIN  PNR XU4G3L            PNR XU4G3L checked in, boarding pass flight AC101 seat 1A
  [t=16 ] REQ 7  CHECKIN  PNR R2VEEP            PNR R2VEEP checked in, boarding pass flight AC101 seat 1C
  [t=22 ] REQ 9  CHECKIN  PNR 89CA7R            PNR 89CA7R checked in, boarding pass flight AC101 seat 1D
  [t=25 ] REQ 4  CANCEL   PNR L7D28G            PNR L7D28G cancelled, seat 1B released
  [t=34 ] REQ 8  CANCEL   PNR 89CA7R            FAILED: PNR 89CA7R is CHECKED_IN (needs BOOKED)
  [t=40 ] REQ 12 BOOK     pax 3 AC303 2D        PNR UK7NRY seat 2D confirmed, charged $610
  [t=49 ] REQ 6  BOOK     pax 8 AC202 5C        PNR Y3W3UU seat 5C confirmed, charged $420
  [t=53 ] REQ 13 BOOK     pax 4 AC202 2B        PNR ABMVFR seat 2B confirmed, charged $420
  [t=54 ] REQ 5  BOOK     pax 7 AC101 3A        PNR ADQCEM seat 3A confirmed, charged $250
  [t=57 ] REQ 10 BOOK     pax 1 AC202 1A        PNR TNPCUB seat 1A confirmed, charged $420
  [t=81 ] REQ 11 BOOK     pax 2 AC303 2D        FAILED: seat already taken
  [t=93 ] REQ 1  BOOK     pax 6 AC101 3A        FAILED: seat already taken
  [t=103] REQ 2  REFUND   PNR JCTGWM            PNR JCTGWM refunded $200 (20% cancellation fee kept)

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  BOOK     -     0    24         2     0     93   69   93    0
  2  REFUND   -     1    28         4     4    103   74  102    3
  3  CHECKIN  yes   2     2         1     2      4    0    2    0
  4  CANCEL   -     3     3         4     8     25   19   22    5
  5  BOOK     -     4     6         3    10     54   44   50    6
  6  BOOK     -     5     5         3    12     49   39   44    7
  7  CHECKIN  -     6     2         2    14     16    8   10    8
  8  CANCEL   -     8     3         4    18     34   23   26   10
  9  CHECKIN  -     9     2         2    20     22   11   13   11
 10  BOOK     -    10     7         3    22     57   40   47   12
 11  BOOK     -    12    14         3    25     81   55   69   13
 12  BOOK     -    14     4         3    29     40   22   26   15
 13  BOOK     yes  50     3         1    50     53    0    3    0
```

Output 3b: commit order and per-request table. REQ 5 (burst 6) beats REQ 1 (burst 24) for seat 3A; REQ 12 beats REQ 11 for seat 2D; the check-in (REQ 9) finishes before the cancel (REQ 8), so the cancel fails. Wait = Finish - Arr - Burst, TAT = Finish - Arr, Resp = First - Arr.

terminal: output (run, continued: Gantt chart, queue changes, metrics)

```
Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1  |P3  |P2  |P1  |P4  |P5  |P6  |P7  |P2  |P8  |P9  |P10 |P4  |P11 |P5  |P12 |P6  |P8  |P10 |
|Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |Q1  |
0    2    4    6    8    10   12   14   16   18   20   22   24   25   27   29   31   33   34   36

|P11 |P12 |P1      |P2      |P6  |P5  |P13   |P5  |P10   |P11     |P1      |P2      |
|Q1  |Q1  |Q1      |Q1      |Q2  |Q2  |Q1    |Q2  |Q2    |Q2      |Q2      |Q2      |
36   38   40       44       48   49   50     53   54     57       61       65       69

|P1          |P11         |P2          |P1          |P2             |P2  |
|Q3          |Q3          |Q3          |Q4          |Q4             |Q5  |
69           75           81           87           93              101  103

Queue transitions / preemptions:
  t=8   P1 Q1->Q2 demoted (used 4 ticks)
  t=18  P2 Q1->Q2 demoted (used 4 ticks)
  t=29  P5 Q1->Q2 demoted (used 4 ticks)
  t=32  P1 Q2->Q1 promoted (aged 25 ticks)
  t=33  P6 Q1->Q2 demoted (used 4 ticks)
  t=36  P10 Q1->Q2 demoted (used 4 ticks)
  t=38  P11 Q1->Q2 demoted (used 4 ticks)
  t=42  P2 Q2->Q1 promoted (aged 25 ticks)
  t=44  P1 Q1->Q2 demoted (used 4 ticks)
  t=48  P2 Q1->Q2 demoted (used 4 ticks)
  t=50  P5 preempted by P13 (Q1/RR)
  t=61  P11 Q2->Q3 demoted (used 4 ticks)
  t=65  P1 Q2->Q3 demoted (used 4 ticks)
  t=69  P2 Q2->Q3 demoted (used 4 ticks)
  t=75  P1 Q3->Q4 demoted (used 6 ticks)
  t=87  P2 Q3->Q4 demoted (used 6 ticks)
  t=101 P2 Q4->Q5 demoted (used 8 ticks)

MLFQ-5Q: avg wait 31.08 | avg turnaround 39.00 | avg response 6.92 | throughput 0.126 req/tick | CPU 100.0%
promotions 2, demotions 14, preemptions 1
```

Output 3c: Gantt chart (row 1 request, row 2 queue, row 3 start tick), queue transitions and metrics. Long jobs sink from Q1 to Q5; REQ 1 and REQ 2 are promoted by aging at t=32 and t=42; urgent REQ 13 preempts REQ 5 at t=50.

terminal: output (compare)

```
aerocore> compare
Same 13 requests through each scheduler:
Algorithm   AvgWait  AvgTAT  AvgResp  Throughput  CPU%
MLFQ-5Q       31.08   39.00     6.92       0.126  100.0
FCFS          53.08   61.00    53.08       0.126  100.0
SJF           29.15   37.08    29.15       0.126  100.0
RR(q=2)       34.38   42.31     8.38       0.126  100.0
```

Output 3d: the same 13 requests through MLFQ, FCFS, SJF and RR. MLFQ has the best response time (6.92); SJF has the best waiting time (29.15) with MLFQ close behind (31.08).

Experiment 4: tick-by-tick trace (sched.c)

input_trace.txt (passed to ./aerocore)

```
$ cat input_trace.txt
pax add 0300-1 Ali
pax add 0300-2 Bea
book 1 AC101 1A b=20
book 2 AC101 2A @6 b=1
trace on
run
quit
```

Input 4: b=20 sets a 20-tick burst, @6 makes REQ 2 arrive at tick 6, trace on prints every tick

terminal: output (trace on)

```
aerocore> pax add 0300-1 Ali
passenger #1 created
aerocore> pax add 0300-2 Bea
passenger #2 created
aerocore> book 1 AC101 1A b=20
queued REQ 1: BOOK     pax 1 AC101 1A         arrival t=0   burst 20 prio 3
aerocore> book 2 AC101 2A @6 b=1
queued REQ 2: BOOK     pax 2 AC101 2A         arrival t=6   burst 1  prio 3
aerocore> trace on
aerocore> run
--- trace: tick | running | queues (P<id>:remaining burst) ---
t=0   run P1  Q1/RR   rem 19 | Q1[P1:19] Q2[] Q3[] Q4[] Q5[]
t=1   run P1  Q1/RR   rem 18 | Q1[P1:18] Q2[] Q3[] Q4[] Q5[]
t=2   run P1  Q1/RR   rem 17 | Q1[P1:17] Q2[] Q3[] Q4[] Q5[]
t=3   run P1  Q1/RR   rem 16 | Q1[] Q2[P1:16] Q3[] Q4[] Q5[]
t=4   run P1  Q2/SJF  rem 15 | Q1[] Q2[P1:15] Q3[] Q4[] Q5[]
t=5   run P1  Q2/SJF  rem 14 | Q1[] Q2[P1:14] Q3[] Q4[] Q5[]
t=6   run P2  Q1/RR   rem 0  | Q1[] Q2[P1:14] Q3[] Q4[] Q5[]
t=7   run P1  Q2/SJF  rem 13 | Q1[] Q2[P1:13] Q3[] Q4[] Q5[]
t=8   run P1  Q2/SJF  rem 12 | Q1[] Q2[] Q3[P1:12] Q4[] Q5[]
t=9   run P1  Q3/PRIO rem 11 | Q1[] Q2[] Q3[P1:11] Q4[] Q5[]
t=10  run P1  Q3/PRIO rem 10 | Q1[] Q2[] Q3[P1:10] Q4[] Q5[]
t=11  run P1  Q3/PRIO rem 9  | Q1[] Q2[] Q3[P1:9] Q4[] Q5[]
t=12  run P1  Q3/PRIO rem 8  | Q1[] Q2[] Q3[P1:8] Q4[] Q5[]
t=13  run P1  Q3/PRIO rem 7  | Q1[] Q2[] Q3[P1:7] Q4[] Q5[]
t=14  run P1  Q3/PRIO rem 6  | Q1[] Q2[] Q3[] Q4[P1:6] Q5[]
t=15  run P1  Q4/SRTF rem 5  | Q1[] Q2[] Q3[] Q4[P1:5] Q5[]
t=16  run P1  Q4/SRTF rem 4  | Q1[] Q2[] Q3[] Q4[P1:4] Q5[]
t=17  run P1  Q4/SRTF rem 3  | Q1[] Q2[] Q3[] Q4[P1:3] Q5[]
t=18  run P1  Q4/SRTF rem 2  | Q1[] Q2[] Q3[] Q4[P1:2] Q5[]
t=19  run P1  Q4/SRTF rem 1  | Q1[] Q2[] Q3[] Q4[P1:1] Q5[]
t=20  run P1  Q4/SRTF rem 0  | Q1[] Q2[] Q3[] Q4[] Q5[]
```

Output 4a: P1 is demoted from Q1 to Q2 after 4 ticks (t=3), and at t=6 the new request P2 arrives in Q1 and takes the CPU from P1

```
terminal: output (run, continued)

Committing in scheduler completion order:
  [t=7  ] REQ 2  BOOK     pax 2 AC101 2A         PNR ESEDXE seat 2A confirmed, charged $250
  [t=21 ] REQ 1  BOOK     pax 1 AC101 1A         PNR GAH4JT seat 1A confirmed, charged $250

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  BOOK     -     0    20         3     0     21    1   21    0
  2  BOOK     -     6     1         3     6      7    0    1    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1      |P1  |P2  |P1  |P1          |P1          |
|Q1      |Q2  |Q1  |Q2  |Q3          |Q4          |
0        4    6    7    9           15           21

Queue transitions / preemptions:
  t=4   P1 Q1->Q2 demoted (used 4 ticks)
  t=6   P1 preempted by P2 (Q1/RR)
  t=9   P1 Q2->Q3 demoted (used 4 ticks)
  t=15  P1 Q3->Q4 demoted (used 6 ticks)

MLFQ-5Q: avg wait 0.50 | avg turnaround 11.00 | avg response 0.00 | throughput 0.095 req/tick | CPU 100.0%
promotions 0, demotions 3, preemptions 1
```

Output 4b: P2 commits first (t=7), then P1 (t=21). P1 sinks through Q2, Q3 and Q4.

Experiment 5: booking life cycle, history and transactions (airline.c)

```
input_life.txt (passed to ./aerocore, PNRs from experiment 2)

$ cat input_life.txt
pnr BGLH29
checkin BGLH29
run
seats AC101
cancel BGLH29
run
cancel XG85ZK
run
refund XG85ZK
run
history 2
txns
quit
```

Input 5: commands passed to the program, using the PNRs created in experiment 2

```
terminal: output (pnr, checkin, seats)

AeroCore | aerocore.journal: 7 records recovered, 0 torn/damaged discarded | 'help' for commands
aerocore> pnr BGLH29
Passenger #1 Ayesha Khan
  PNR BGLH29  AC101 seat 3B  BOOKED     fare $250
      tx#2    10-27 05:23:10  BOOK     +250
aerocore> checkin BGLH29
queued REQ 1: CHECKIN  PNR BGLH29             arrival t=0   burst 2  prio 2
aerocore> run

Committing in scheduler completion order:
  [t=2  ] REQ 1  CHECKIN  PNR BGLH29             PNR BGLH29 checked in, boarding pass flight AC101 seat 3B

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  CHECKIN  -     0     2         2     0      2    0    2    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1  |
|Q1  |
0    2

Queue transitions / preemptions:

MLFQ-5Q: avg wait 0.00 | avg turnaround 2.00 | avg response 0.00 | throughput 0.500 req/tick | CPU 100.0%
promotions 0, demotions 0, preemptions 0
aerocore> seats AC101
AC101 KHI->DXB  2026-11-01 08:30  fare $250  (1/40 taken)
      A  B  C  D
  1  [.][.][.][.]
  2  [.][.][.][.]
  3  [.][#][.][.]
  4  [.][.][.][.]
  5  [.][.][.][.]
  6  [.][.][.][.]
  7  [.][.][.][.]
  8  [.][.][.][.]
  9  [.][.][.][.]
 10  [.][.][.][.]
.=free  X=booked  #=checked in
```

Output 5a: the program starts by recovering 7 records from the journal (state persisted). pnr shows the booking; checkin commits at t=2 and seat 3B becomes #.

```
terminal: output (cancel after check-in fails, cancel a booked ticket)

aerocore> cancel BGLH29
queued REQ 1: CANCEL   PNR BGLH29            arrival t=0   burst 3  prio 4
aerocore> run

Committing in scheduler completion order:
  [t=3  ] REQ 1  CANCEL   PNR BGLH29            FAILED: PNR BGLH29 is CHECKED_IN (needs BOOKED)

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  CANCEL   -     0     3         4     0      3    0    3    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1    |
|Q1    |
0      3

Queue transitions / preemptions:

MLFQ-5Q: avg wait 0.00 | avg turnaround 3.00 | avg response 0.00 | throughput 0.333 req/tick | CPU 100.0%
promotions 0, demotions 0, preemptions 0
aerocore> cancel XG85ZK
queued REQ 1: CANCEL   PNR XG85ZK            arrival t=0   burst 3  prio 4
aerocore> run

Committing in scheduler completion order:
  [t=3  ] REQ 1  CANCEL   PNR XG85ZK            PNR XG85ZK cancelled, seat 1A released

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  CANCEL   -     0     3         4     0      3    0    3    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1    |
|Q1    |
0      3

Queue transitions / preemptions:

MLFQ-5Q: avg wait 0.00 | avg turnaround 3.00 | avg response 0.00 | throughput 0.333 req/tick | CPU 100.0%
promotions 0, demotions 0, preemptions 0
```

Output 5b: cancelling the checked-in booking fails ("is CHECKED_IN (needs BOOKED)"); cancelling the other booking succeeds and releases seat 1A.

```
terminal: output (refund, history, txns)

aerocore> refund XG85ZK
queued REQ 1: REFUND   PNR XG85ZK            arrival t=0   burst 8  prio 5
aerocore> run

Committing in scheduler completion order:
  [t=8  ] REQ 1  REFUND   PNR XG85ZK            PNR XG85ZK refunded $336 (20% cancellation fee kept)

 ID  Type     Urg Arr Burst Prio(end) First Finish Wait  TAT Resp
  1  REFUND   -     0     8         5     0      8    0    8    0

Gantt chart  (row 1: request, row 2: queue, row 3: tick)
|P1      |P1      |
|Q1      |Q2      |
0        4        8

Queue transitions / preemptions:
  t=4   P1 Q1->Q2 demoted (used 4 ticks)

MLFQ-5Q: avg wait 0.00 | avg turnaround 8.00 | avg response 0.00 | throughput 0.125 req/tick | CPU 100.0%
promotions 0, demotions 1, preemptions 0
aerocore> history 2
Passenger #2 Bilal Ahmed (0300-7654321)
  PNR XG85ZK  AC202 seat 1A  REFUNDED   fare $420
      tx#1    10-27 05:23:10  BOOK     +420
      tx#4    10-27 05:23:21  CANCEL   +0
      tx#5    10-27 05:23:21  REFUND   -336
aerocore> txns
  XG85ZK      tx#1    10-27 05:23:10  BOOK     +420
  BGLH29      tx#2    10-27 05:23:10  BOOK     +250
  BGLH29      tx#3    10-27 05:23:21  CHECKIN  +0
  XG85ZK      tx#4    10-27 05:23:21  CANCEL   +0
  XG85ZK      tx#5    10-27 05:23:21  REFUND   -336
```

Output 5c: the refund returns $336, which is 80% of $420. history and txns show the BOOK, CANCEL and REFUND transactions.

Experiment 6: journal crash and recovery (airline.c)

```
input (two runs of ./aerocore)

$ ./aerocore
demo
crash 2
run
$ ./aerocore   # restart
txns
quit
```

Input 6: arm the crash, run a batch, then restart the program

```
terminal: output (crash 2, then restart)

AeroCore | aerocore.journal: 0 records recovered, 0 torn/damaged discarded | 'help' for commands
aerocore> demo
queued REQ 1: BOOK     pax 6 AC101 3A         arrival t=0   burst 24 prio 3
queued REQ 2: REFUND   PNR G6GVJH             arrival t=1   burst 28 prio 5
queued REQ 3: CHECKIN  PNR PMXHPH             arrival t=2   burst 2  prio 1 URGENT
queued REQ 4: CANCEL   PNR XF2AA3             arrival t=3   burst 3  prio 4
queued REQ 5: BOOK     pax 7 AC101 3A         arrival t=4   burst 6  prio 3
queued REQ 6: BOOK     pax 8 AC202 5C         arrival t=5   burst 5  prio 3
queued REQ 7: CHECKIN  PNR MLLNL9             arrival t=6   burst 2  prio 2
queued REQ 8: CANCEL   PNR SK98GR             arrival t=8   burst 3  prio 4
queued REQ 9: CHECKIN  PNR SK98GR             arrival t=9   burst 2  prio 2
queued REQ 10: BOOK     pax 1 AC202 1A         arrival t=10  burst 7  prio 3
queued REQ 11: BOOK     pax 2 AC303 2D         arrival t=12  burst 14 prio 3
queued REQ 12: BOOK     pax 3 AC303 2D         arrival t=14  burst 4  prio 3
queued REQ 13: BOOK     pax 4 AC202 2B         arrival t=50  burst 3  prio 1 URGENT
demo batch queued: try 'run', then 'compare'
aerocore> crash 2
armed: journal write #3 from now will be torn. 'run' a batch to trigger it.
aerocore> run

Committing in scheduler completion order:
  [t=4  ] REQ 3  CHECKIN  PNR PMXHPH             PNR PMXHPH checked in, boarding pass flight AC101 seat 1A
  [t=16 ] REQ 7  CHECKIN  PNR MLLNL9             PNR MLLNL9 checked in, boarding pass flight AC101 seat 1C
*** simulated power loss mid-write ***
(process exits with status 137)
$ ./aerocore   # restart
AeroCore | aerocore.journal: 19 records recovered, 1 torn/damaged discarded | 'help' for commands
aerocore> txns
  PMXHPH      tx#1    10-27 05:02:30  BOOK     +250
  XF2AA3      tx#2    10-27 05:02:30  BOOK     +250
  MLLNL9      tx#3    10-27 05:02:30  BOOK     +250
  SK98GR      tx#4    10-27 05:02:30  BOOK     +250
  G6GVJH      tx#5    10-27 05:02:30  BOOK     +250
  G6GVJH      tx#6    10-27 05:02:30  CANCEL   +0
  PMXHPH      tx#7    10-27 05:02:30  CHECKIN  +0
  MLLNL9      tx#8    10-27 05:02:30  CHECKIN  +0
```

Output 6: two commits succeed, the third write is torn and the process exits with status 137. After the restart: 19 records recovered, 1 torn record discarded, and the two earlier check-ins (tx#7, tx#8) are present. The transaction that was being written never appears.

5. Result and conclusion

The five-queue MLFQ answered requests almost as fast as Round Robin and waited almost as little as SJF, and the journal recovered cleanly from a simulated power loss.

Findings

- **Build and tests:** both programs compile without warnings and `make test` prints ALL TESTS PASSED (Output 1).
- **Scheduler decides the winner:** in the 13-request demo, REQ 5 (burst 6, arrived t=4) took seat 3A at t=54, while REQ 1 (burst 24, arrived t=0) failed at t=93. REQ 12 beat REQ 11 for seat 2D, and a check-in finished before a cancel on the same PNR, so the cancel failed (Output 3b).
- **Queue behaviour:** the batch finished in 103 ticks with 14 demotions, 2 aging promotions and 1 preemption (urgent REQ 13 took the CPU from REQ 5 at t=50). Both urgent requests waited 0 ticks (Output 3c).
- **Comparison:** average response time was 6.92 ticks for MLFQ, against 8.38 for RR, 29.15 for SJF and 53.08 for FCFS. Average waiting time was 29.15 for SJF, 31.08 for MLFQ, 34.38 for RR and 53.08 for FCFS. Throughput (0.126 requests per tick) and CPU use (100%) were identical (Output 3d).
- **Aging:** in the test workload a long job waited more than 30 ticks with aging off, and at most 20 ticks with the aging limit at 15.
- **Recovery:** after `crash 2` the third journal write was torn. On restart 19 records were recovered, 1 torn record was discarded, and the two earlier commits were intact (Output 6).

Conclusion

Scheduling policy has a visible effect on a real outcome: who gets a seat. MLFQ favours short, urgent requests, so a long request that arrives first can lose a seat race. That is the price of its low response time, and aging keeps long requests from starving. Writing every change as one checksummed, fsync'd journal line made crash recovery simple and correct.

Limits: time is simulated, context switches are free, capacities are fixed, the journal is never compacted and there are no concurrent writers. Natural extensions are journal compaction and running the scheduler against real concurrent clients.